HStore and HyperGraphDB: a storage-level benchmark of two persistent hypergraph engines

Metacog Labs

HStore 0.1.0, commit 9291e18

October 2026

Abstract

We compare HStore, a persistent hypergraph storage engine built on copy-on-write counted B+trees, with HyperGraphDB 1.4 on Berkeley DB Java Edition. Both systems run 26 storage-level workloads on the same skewed synthetic hypergraph, with up to 1.45 million incidences and equal 64 MiB caches, in three configurations: two data sizes and two commit modes. Every read result is cross-checked between the systems. At scale 1 with asynchronous commits, HStore enumerates incidence sets 4.21× faster, counts shared hyperedges 11.3× faster, applies batched updates 5.38× faster, and needs 3.75× less disk. These leads come from storing incidence sets as small sorted trees, and from commits that log only the keys they changed while a background checkpoint writes pages. At scale 4, where both systems' files outgrow their caches, these leads hold or widen, and HStore enumerates hyperedge members 3.22× faster, because its cached catalog pages decode only the records that are read. HyperGraphDB recovers from a crash faster at scale 1. With synchronous commits, both systems are bound by the cost of one forced write per commit. We also report latency percentiles, bytes written, heap use, space reclamation and crash recovery, the variation over runs, and a durability setting in HyperGraphDB that, left unchecked, makes its synchronous commits unsynced.

1 Introduction

Many datasets record facts that involve more than two things at once. An insurance claim joins a patient, a provider, a drug and a pharmacy. A paper joins its authors, and a meeting joins its participants. A hypergraph stores each such fact as one hyperedge over all of its participants. A property graph has to break the same fact into a new node and one edge per participant, and a relational schema into a join table. Queries then ask hypergraph questions: which facts involve this entity, which facts involve both of these entities, and what lies two hops away.

HyperGraphDB [1] is the established open-source hypergraph database for the JVM. It stores atoms and links in Berkeley DB Java Edition (JE), a general-purpose log-structured key-value store. HStore is a newer engine built for this data model. Its incidence sets and hyperedge memberships are counted B+trees, and its storage, transaction and recovery design is described in section 2.

This report compares the two systems at the storage level, on the same machine and data, with the same cache budget. We measure what an embedded application sees: throughput of loads, updates, deletes and reads; per-operation latency, including tail latency while another thread writes; behaviour of giant hyperedges; disk footprint before and after deletes and space reclamation; time to recover after a crash; and the bytes written, heap allocated and GC time behind each result. Every read result is cross-checked between the two systems.

Section 3 describes the dataset, workloads and measurement method. Section 4 presents the results, and section 5 explains them in terms of each system's design. Section 6 lists the limits of this study. Full tables, per-run values and the steps to reproduce every number are in the appendices.

Summary of findings.

- **Incidence-based reads** are faster in HStore in every configuration: 4.21× to 11.4× for incidence sets, 11.3× to 19.7× for co-membership counts, and 28.0× to 88.9× directly after a reopen.
- **Writes** cost HStore far fewer bytes. Single-update commits write 659 B against 6.9 KiB, and batched updates run 5.38× to 33.1× faster. Bulk insertion of hyperedges is close at scale 1 (1.25× to 1.43×) and 10.3× faster at scale 4.
- **Member enumeration** is 3.22× faster in HStore at scale 4, where HStore's catalog stays in its node cache because cached leaves decode only the records that are read. At scale 1 the ranges overlap. Two-hop neighbourhoods are 3.34× to 5.53× faster.
- **Tail latency.** HStore's p99 and p99.9 latencies are lower than HyperGraphDB's in every latency workload and configuration, by 1.1 to 35 times.
- **Disk.** HStore's files are 3.45× to 13.9× smaller after loading. After deletes they first grow while retained history still references the replaced pages.
- **Recovery.** Neither system lost an acknowledged commit after SIGKILL. HyperGraphDB opens a crashed store faster at scale 1, and HStore at scale 4.

2 Systems under test

Both systems are embedded Java libraries. Each runs in its own JVM process, and the benchmark harness calls its API directly. Neither is accessed over a network.

2.1 HStore

Data model. Every stored object is an *atom* with a 64-bit identifier. An atom is a node or a hyperedge. A hyperedge has any number of members, and a member can itself be a hyperedge. Hyperedges are either *set* edges, keyed by member, or *ordered* edges, keyed by position. Each membership carries a role set, a weight, a validity interval and a qualifier. The benchmark uses set edges and plain memberships.

Layout. All persistent state is held in copy-on-write counted B+trees. Every child reference stores a summary of its subtree: the entry count, the key range and an order-independent fingerprint. Three trees matter here:

- The *catalog* maps an atom id to its record. A hyperedge record holds a reference to that edge's *membership tree*, keyed by member id. If the membership tree is one leaf of at most 32 entries that fits in about 100 bytes, it is stored inside the record itself and needs no page of its own.
- The *reverse incidence index* maps an atom id to the set of hyperedges that contain it. Sets of up to 48 entries are stored inline in the index leaf. Larger sets are promoted to a tree of their own.
- Secondary indexes on node keys and properties, which the benchmark touches only through node insertion and updates.

Enumerating an incidence set is a cursor walk over one inline list or one small tree. Counting the hyperedges that contain two given atoms is a leapfrog intersection [3] of their two incidence trees. The intersection skips whole subtrees whose key range cannot contain a match.

Pages and files. A tree node is written as a self-describing image: an 80-byte header with a CRC32C checksum, followed by the payload. Images are packed into append-only segment files of 256 MiB, aligned to 64-byte units. A page directory maps each page number to the image's current address, so compaction can move an image without rewriting the trees that refer to it. Reads go through a memory map of each segment. Decoded nodes are kept in a node cache whose budget is set in bytes. In catalog and incidence index leaves, each record is preceded by its length. A leaf read from a page keeps the page bytes and the offset of every record, and decodes a record the first time it is read. A decoded record is then kept in the leaf and charged to the node cache, so a lookup that misses the cache decodes only the record it needs.

Generations and readers. A committed state is a *generation*: an immutable vector of tree roots. A reader pins a generation and reads it without locks. Writers never modify a node that a published generation can reach, as in other copy-on-write B+tree stores [4]. The last 64 generations are retained for time travel.

Commits and checkpoints. Figure 1 shows the write path. A transaction collects its changes in a private workspace. At commit it applies them to new copies of the affected tree paths, appends one write-ahead log record listing the keys it changed in each tree, and publishes the new generation. The new tree nodes stay in memory, reachable from the published roots. A background checkpoint later writes every node that no earlier checkpoint wrote, syncs the files, updates page reference counts, and publishes a catalog image. It then swaps the stored references into the current generation and truncates the log. Only the swap holds the commit lock. Memory held by unwritten nodes is capped at 64 MiB. A commit that finds the cap reached waits for a checkpoint.

Two exceptions write pages before the checkpoint. A large bulk load spills finished subtrees to pages while it runs, and a tree that changed in more than 4,096 places is written to pages at commit and logged by reference. Under *synchronous* durability, a commit returns after its log record is fsynced; concurrent commits share one fsync (group commit). Under *asynchronous* durability the fsync is skipped, but commits are still published strictly in order.

Recovery. On open, HStore loads the newest catalog image and replays the logged key changes of every commit after it, in commit order. A commit's changes are applied only if its commit record reached the log, so recovery always yields a prefix of the commit order.

Space reclamation. Every page has a reference count, which changes only at checkpoints. A page no generation can reach is released once no pinned reader can see it. A segment whose live bytes fall below half of the bytes written to it is compacted: its live images are copied to the active segment, the page directory is updated, and the old file is deleted once no reader can still be using it. A background maintenance pass, run every 500 ms, compacts one such segment at a time. An explicit compaction request compacts all of them.

2.2 HyperGraphDB

HyperGraphDB [1] is a general hypergraph database for Java. Its atoms are identified by persistent handles (UUIDs). A link stores the ordered array of its target handles. The benchmark uses `HGPlainLink` for hyperedges and `String` atoms for nodes. The incidence set of an atom, the links pointing to it, is maintained by HyperGraphDB in a separate index. The atom cache keeps recently used atoms and incidence sets in memory as Java objects.

We use the Berkeley DB Java Edition storage implementation (`bdb-je`), the default for HyperGraphDB 1.4. JE is a log-structured B+tree store [2, 6]: every update appends a new version of the changed records and internal nodes to the log. A background cleaner copies the live entries out of underused log files and deletes them. A checkpointer bounds recovery time. Incidence sets are stored as sorted duplicate

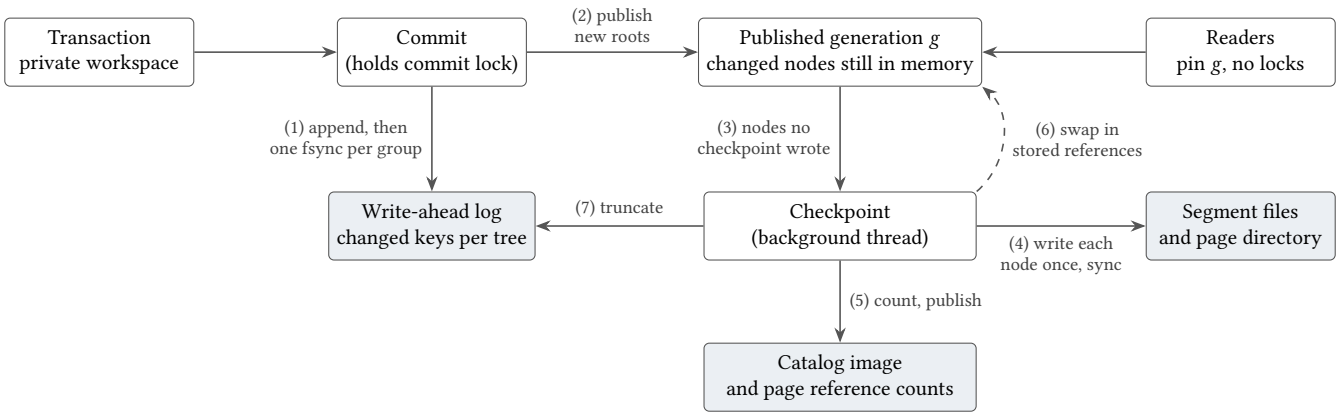


Figure 1: HStore's write path. (1, 2) A commit logs only the keys it changed and publishes new roots whose changed nodes are still in memory. (3–7) A background checkpoint writes those nodes to pages, once for all the commits since the previous checkpoint, then swaps the stored references into the current generation and truncates the log. Only step 6 takes the commit lock.

sets in a JE database. A lookup opens a cursor at the atom's handle.

2.3 Synchronous commits in HyperGraphDB

HyperGraphDB 1.4 begins every JE transaction with an explicit `WRITE_NO_SYNC` durability. That setting overrides the environment-wide `COMMIT_SYNC`, so configuring the environment alone leaves commits unsynced: in a first run of the synchronous configuration, HyperGraphDB committed single updates in about 0.1 ms, while one forced write on this machine takes about 3.8 ms. For the synchronous configuration the HyperGraphDB adapter therefore calls `Environment.flushLog(true)` after every commit. This writes and forces JE's log, which is what `COMMIT_SYNC` would do at commit time. The asynchronous configuration is unaffected.

2.4 Versions and configuration

Table 1 lists the configuration of both systems. The two caches are given the same 64 MiB budget. HStore's budget covers decoded nodes. JE's budget covers its B+tree nodes and records, and HyperGraphDB's atom cache comes on top of it. That cache is sized by HyperGraphDB itself and cannot be capped in bytes. The retained-heap figure in section 4.7 shows what each store actually kept.

3 Benchmark design

3.1 Dataset

The dataset is generated from a fixed seed (20260904), so both systems load exactly the same graph. At scale s it has 50,000 s nodes and 100,000 s hyperedges. The cardinality of a hyperedge follows a geometric tail: it starts at 2 and each further member is added with probability 0.62, up to 32, for a mean of 3.6. Members are drawn as $\lfloor n\,u^{2.2} \rfloor$ for uniform $u \in [0, 1)$ over n nodes. Low-numbered nodes therefore become hubs with large incidence sets, as in co-authorship, claims or citation data. Hyperedges never repeat a member.

Table 2 gives the sizes and the skew. The largest incidence set at scale 1 holds 2,645 hyperedges, while half of all nodes are in 5 or fewer. Only 370 incidence sets exceed the 48 entries that HStore stores inline. Probe sets drawn from the same seed hold 100,000 random nodes and hyperedges, 20,000 node pairs taken from a common hyperedge (so that co-membership counts are non-zero), and 20,000 node updates. A separate churn set, drawn after everything else, selects 20% of the hyperedges for deletion and 20,000 other hyperedges, each with at least three members, that lose one member.

3.2 Workloads

Table 3 defines the workloads. They run in the order listed, in one process per system and run, against one database directory. The table also names the API call each system uses. Each adapter uses the plain storage calls of its system and its own identifier type: `long` ids for HStore and persistent handles for HyperGraphDB. Higher-level traversal frameworks, such as HyperGraphDB's breadth-first traversal, are not used, so the numbers reflect the storage layer.

HyperGraphDB links are immutable, so `churn.remove` replaces each link with a new one that lacks the removed member. `read.twohop` runs before `large.ingest`, because once a hyperedge containing every node exists, every node is two hops from all others. `recover` runs in a child JVM with the same flags. The child ingests nodes in transactions of 1,000 into a fresh directory and prints the count after each acknowledged commit. The parent kills it with SIGKILL once half the nodes are acknowledged, then times opening the directory and counts the surviving nodes.

3.3 Measurement

Throughput. Each workload is timed with `System.nanoTime()` around its whole operation set. Throughput is operations divided by elapsed time. Each write

Table 1: System configuration. Both systems use their defaults except where stated.

	HStore	HyperGraphDB
Version	0.1.0, commit `9291e18`, storage format 8	1.4, commit `99485a1`; JE 5.0.73
Cache	node cache, 64 MiB	JE cache 64 MiB, plus atom cache
Async commits	no fsync, ordered publication	JE `WRITE_NO_SYNC` (default)
Sync commits	fsync per commit group	JE log forced after each commit (section 2.3)
Page / node size	16 KiB pages	JE defaults
Space reclamation	compact segments below 50% live	cleaner, 50% minimum utilisation
History	64 generations	none

Table 2: Dataset statistics, computed from the generator. An incidence set is the set of hyperedges that contain a node.

	Scale 1	Scale 4
Nodes	50,000	200,000
Hyperedges	100,000	400,000
Incidences	362,757	1,453,427
Hyperedge cardinality, mean / p99 / max	3.63 / 11 / 30	3.63 / 11 / 30
Incidence set size, mean	7.26	7.27
Incidence set size, p50 / p90 / p99	5 / 12 / 41	5 / 13 / 42
Incidence set size, p99.9 / max	145 / 2,645	142 / 5,714
Nodes in no hyperedge	575	2,319
Incidence sets above 48 entries	370	1,521

workload ends with a flush outside the timed section: an HStore checkpoint, or `Environment.sync()` for JE. Bytes still in buffers are therefore counted against the workload that produced them.

Warm-up. Before every timed read workload except `read.incidence.cold`, the same workload runs untimed for at least 2 s. Both systems get the same warm-up. The numbers describe a process whose JIT has compiled the read path. A short warm-up mostly measures how fast each system's code gets compiled: in our trials, a warm-up over 10% of the probes left `read.members` about three times below its steady rate.

Latency. `latency.read` and `latency.commit` time every transaction separately and report p50, p99, p99.9 and the maximum. In `mixed` and `compaction`, the writer runs on a fixed schedule (open loop), and each commit's latency is measured from the time it was *scheduled* to start. A stall therefore shows up in the latency of every commit queued behind it, instead of silently slowing the writer down. The readers in `mixed.read` run in a closed loop and keep a uniform sample of 200,000 latencies each, plus the exact maximum.

Bytes written and read. Both systems count their own I/O. For HStore this is the log bytes plus the page bytes written, including pages moved by compaction. For HyperGraphDB it is JE's sequential plus random write bytes, including the cleaner. Reads count the bytes each system reads from its files: HStore's segment reads, and JE's sequential plus random read bytes. A read served from the system's own cache counts as zero. A read that reaches a file counts even when the operating system's page cache serves it. Write amplification divides bytes written by an estimate of the logical bytes: 8 bytes per id plus the value, 24 bytes per updated value, $(c+1)\times 8$ bytes for a new or deleted hyperedge of cardinality c, and 16 bytes per removed member.

Memory. For every workload the harness records the bytes allocated, from `ThreadMXBean` (the whole process for write workloads, the reading thread alone for single-threaded reads), and the stop-the-world GC time from `GarbageCollectorMXBean`. After ingest it forces a full GC and records the heap in use, minus a baseline taken before the store opened. This is the *retained heap*.

Correctness cross-check. Every read and delete workload returns a checksum: the number of ids visited, intersected or found. Both systems must return the same checksum in every run, or the result is flagged. All checksums agreed in every run reported here. Workloads whose result depends on timing (`mixed`, `compaction`, `recover`) are not checked.

3.4 Environment and statistics

All runs used one machine: an Apple M5 with 4 performance and 6 efficiency cores, 24 GiB of memory, and a 1 TB Apple SSD formatted with APFS, running macOS 26.5.1. The JVM was Oracle GraalVM for JDK 25.0.4 with the HotSpot JIT, a fixed 4 GiB heap (`-Xms4g -Xmx4g`) and the Parallel collector. Each system ran in its own JVM, and runs alternated between the two systems so that slow periods affect both. We ran three configurations:

- **S1 async**: scale 1, asynchronous commits, 5 runs per system;
- **S1 sync**: scale 1, synchronous commits, 5 runs per system;
- **S4 async**: scale 4, asynchronous commits, 3 runs per system. With 64 MiB caches, the files of both systems are larger than their caches.

Each table reports the median over runs. Ratios are taken between medians and oriented so that values above 1 favour HStore: throughput divides HStore by HyperGraphDB, while times, latencies and sizes divide HyperGraphDB by HStore. Where the ranges of the two systems' runs overlap, the ratio is marked with †, and the difference should not be read as significant. Appendix B gives every run and the coefficient of variation of each workload.

Table 3: Workloads, in execution order. "txn" is a transaction. All read workloads except `read.incidence.cold` run after a warm-up (section 3.3).

Workload	Operation	HStore call	HyperGraphDB call
`ingest.nodes`	insert all nodes, 1,000 per txn	`Writer.node`	`graph.add(String)`
`ingest.edges`	insert all hyperedges, 1,000 per txn	`Writer.edge`, `Writer.load`	`graph.add(HGPlainLink)`
`read.incidence`	enumerate a node's incidence set; 100,000 probes, 1,000 per read txn	`forEachIncidentEdge`	`getIncidenceSet`
`read.members`	enumerate a hyperedge's members; 100,000 probes	`forEachMemberId`	`getTargetAt`
`read.comembership`	count hyperedges containing both nodes of a pair; 20,000 pairs	leapfrog intersection	`hg.and(hg.incident, hg.incident)`
`read.twohop`	count distinct nodes sharing a hyperedge with a node; 10,000 probes	incidence, then members	incidence, then targets
`read.incidence.parallel`	`read.incidence` from 8 threads, 100,000 probes each	as above	as above
`latency.read`	one incidence lookup per read txn, each timed; 100,000 txns	as above	as above
`write.update`	replace a node's value; 20,000 updates, 1,000 per txn	`Writer.set`	`graph.replace`
`latency.commit`	one update per write txn, each timed; 5,000 txns	`Writer.set`	`graph.replace`
`mixed.read`	single incidence lookups from 7 threads for 10 s, beside `mixed.write`	as `latency.read`	as `latency.read`
`mixed.write`	one thread committing 10 updates per txn on a fixed schedule of 1,000 updates/s	`Writer.set`	`graph.replace`
`large.ingest`	create one hyperedge containing every node	`Writer.load`	one `HGPlainLink`
`large.scan`	enumerate the large hyperedge 400 times	`forEachMemberId`	`getTargetAt`
`large.probe`	test whether a node belongs to the large hyperedge; 100,000 probes	membership-tree lookup	`getIncidenceSet(n).contains(e)`
`reopen`	close cleanly and open again	`HypergraphDatabase.open`	`HGEnvironment.get`
`read.incidence.cold`	`read.incidence` immediately after the reopen, no warm-up		
`disk`	bytes on disk after a clean shutdown		
`churn.delete`	delete 20% of the hyperedges, 1,000 per txn	`Writer.delete`	`graph.remove`
`churn.remove`	remove one member from each of 20,000 other hyperedges, 1,000 per txn	`Writer.remove`	`graph.replace` with a new link
`read.incidence.churned`	`read.incidence` after the deletes		
`disk.churned`	bytes on disk after the deletes and a clean shutdown		
`compaction.baseline`	single-update commits at 50/s for 10 s, each timed	`Writer.set`	`graph.replace`
`compaction`	the same commits while a full space reclamation runs, starting 1 s in	`StorageEngine.compact`	`cleanLog` until idle, then checkpoint
`disk.compacted`	bytes on disk after the reclamation and a clean shutdown		
`recover`	open a store whose ingest process was killed with SIGKILL	`HypergraphDatabase.open`	`HGEnvironment.get`

4 Results

Figure 2 shows the ratio for every workload in all three configurations. HStore leads in all but three of them: single-update commits in S1 async, where the ranges overlap, and opening the store after a crash at scale 1, in both commit modes. Table 4 gives the same ratios with the run-range test from section 3.4. Table 5 gives the absolute numbers for the S1 async configuration. The other configurations are tabulated in appendix A.

4.1 Loading, updating and deleting

At scale 1 with asynchronous commits, HStore inserts nodes 1.80× and hyperedges 1.43× as fast as HyperGraphDB. Updates differ more. Replacing node values 1,000 at a time runs 5.38× faster in HStore (69.9 k against 13.0 k updates per second). Deleting hyperedges is 3.10× faster and removing single members 2.47× faster. Building one hyperedge over all 50,000 nodes takes HStore a single bulk load, 5.25× faster than HyperGraphDB.

Single-update transactions are the one write workload where HStore does not lead: 12.4 k against 14.1 k commits per second (0.88×, ranges overlapping). With asynchronous commits, neither system waits for the disk, and the cost of one commit is dominated by work in memory. Section 4.4 shows that the two medians are close and that the difference lies in the tail.

4.2 Reads

HStore's largest margins are on reads that go through incidence sets. It enumerates a node's incidence set 4.21× faster than HyperGraphDB, counts the hyperedges shared by two nodes 11.3× faster, and computes two-hop neighbourhoods 5.53× faster. With 8 threads, incidence lookups reach 36.56 M per second in HStore against 2.25 M (16.3×). When every lookup is its own read transaction (`latency.read`), the ratio is 8.39×. Enumerating the members of a hyperedge, the read where HyperGraphDB's atom cache helps most, is 2.30×

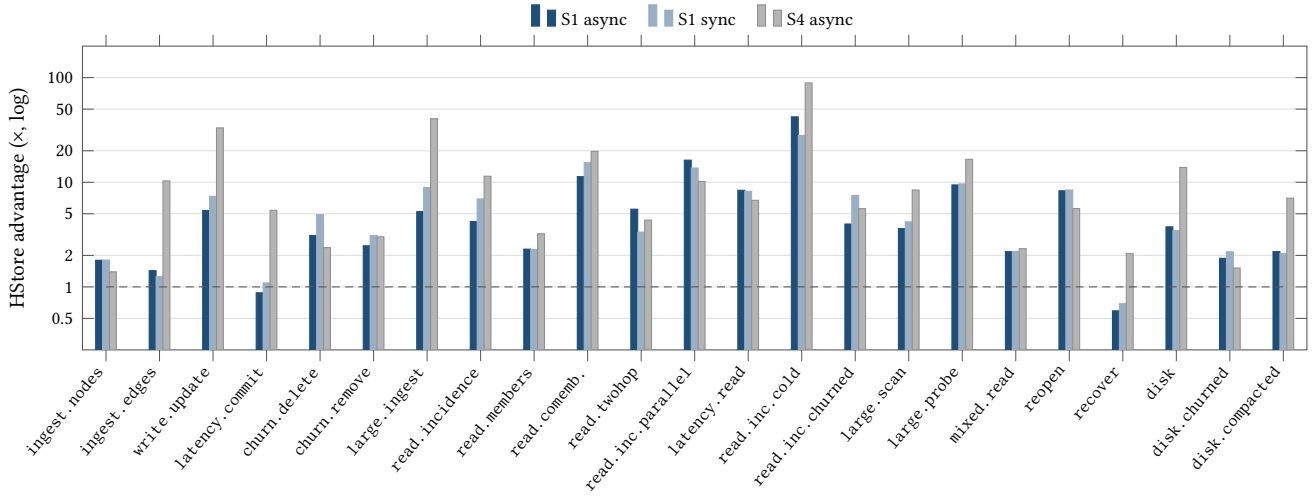


Figure 2: Ratio of HStore to HyperGraphDB for every workload and configuration (log scale). Bars above the dashed line favour HStore. Throughput workloads divide HStore by HyperGraphDB; `reopen`, `recover` and the disk sizes divide HyperGraphDB by HStore.

Table 4: Ratio of medians, HStore over HyperGraphDB, oriented so that values above 1 favour HStore (bold). †: the run ranges of the two systems overlap.

Workload	S1 async	S1 sync	S4 async
`ingest.nodes`	**1.80×**	**1.81×**†	**1.39×**†
`ingest.edges`	**1.43×**	**1.25×**†	**10.3×**
`write.update`	**5.38×**	**7.34×**	**33.1×**
`latency.commit`	0.88×†	**1.09×**	**5.39×**
`churn.delete`	**3.10×**	**4.91×**	**2.38×**
`churn.remove`	**2.47×**	**3.10×**	**3.01×**
`large.ingest`	**5.25×**	**8.93×**	**40.5×**
`read.incidence`	**4.21×**	**6.93×**	**11.4×**
`read.members`	**2.30×**†	**2.28×**†	**3.22×**
`read.comembership`	**11.3×**	**15.4×**	**19.7×**
`read.twohop`	**5.53×**	**3.34×**	**4.35×**
`read.incidence.parallel`	**16.3×**	**13.7×**	**10.2×**
`latency.read`	**8.39×**	**8.21×**	**6.73×**
`read.incidence.cold`	**42.2×**	**28.0×**	**88.9×**
`read.incidence.churned`	**3.99×**	**7.47×**	**5.59×**
`large.scan`	**3.61×**	**4.18×**†	**8.43×**
`large.probe`	**9.43×**	**9.65×**	**16.6×**
`mixed.read`	**2.17×**	**2.18×**	**2.32×**
`reopen`	**8.29×**	**8.44×**	**5.61×**
`recover`	0.59×	0.69×	**2.08×**
`disk`	**3.75×**	**3.45×**	**13.9×**
`disk.churned`	**1.88×**	**2.17×**	**1.52×**
`disk.compacted`	**2.17×**	**2.09×**	**7.05×**

faster in HStore at the median, with overlapping ranges: one of the five HStore runs was slower than HyperGraphDB's median.

Two read workloads run under special conditions. Directly after a reopen, before any warm-up, HStore serves incidence lookups at 4.04 M per second against 95.9 k (42.2×). HStore read 27 B per lookup from its files in that phase, against 3.2 KiB for JE. After the deletes, incidence lookups are 3.99× faster.

4.3 Giant hyperedges

On the hyperedge that contains all 50,000 nodes, HStore scans members 3.61× faster (4.05 G members per second against 1.12 G) and tests membership 9.43× faster. A membership test in HStore is one descent of the edge's membership tree.

4.4 Latency

Figure 3 and table 6 give the latency distributions. For single-update commits with asynchronous commits, the medians are close, with HyperGraphDB slightly ahead: 52 µs for HStore and 45 µs for HyperGraphDB. The tails favour HStore. At p99, HStore needs 296 µs and HyperGraphDB 466 µs. At p99.9 the figures are 441 µs and 936 µs. Read transactions take 0.2 µs at the median in HStore and 0.9 µs in HyperGraphDB, and 1.5 µs against 23 µs at p99.9.

Reads beside a writer. In `mixed`, the writer's median rate was 1,000 updates per second in both systems; its slowest run reached 1,000 updates per second in HStore and 999 in HyperGraphDB. Meanwhile the 7 readers completed 2.46 M lookups per second in HStore and 1.13 M in HyperGraphDB (2.17×). Reader latency at p99.9 was 26 µs against 305 µs. The writer's commits, measured from their scheduled start, took 9.65 ms against 16.94 ms at p99.

Commits during space reclamation. In the `compaction` window, the median commit latency was about the same with and without a reclamation running, for both systems. At p99, HStore's commits took 5.44 ms during compaction against 5.43 ms without; HyperGraphDB's took 11.59 ms during cleaning against 7.50 ms without.

4.5 Disk footprint, deletes and reclamation

After loading at scale 1, HStore's files take 92.4 MiB and HyperGraphDB's 346.4 MiB (fig. 4). After the deletes the figures

Table 5: S1 async: median throughput (operations per second), times and sizes over 5 runs per system.

Workload	Operation	HStore	HyperGraphDB	Ratio	Agree
`ingest.nodes`	Insert nodes (1,000 per txn)	67.7 k	37.7 k	**1.80**×	yes
`ingest.edges`	Insert hyperedges (1,000 per txn)	50.0 k	35.0 k	**1.43**×	yes
`write.update`	Update node values (1,000 per txn)	69.9 k	13.0 k	**5.38**×	yes
`latency.commit`	Single-update transactions	12.4 k	14.1 k	0.88×	yes
`churn.delete`	Delete hyperedges (1,000 per txn)	24.5 k	7.9 k	**3.10**×	yes
`churn.remove`	Remove one member (1,000 per txn)	43.2 k	17.5 k	**2.47**×	yes
`large.ingest`	Build one giant hyperedge	610.1 k	116.3 k	**5.25**×	yes
`read.incidence`	Incidence set of a node	6.26 M	1.49 M	**4.21**×	yes
`read.members`	Members of a hyperedge	6.11 M	2.66 M	**2.30**×	yes
`read.comembership`	Co-membership count of a pair	610.3 k	53.9 k	**11.3**×	yes
`read.twohop`	Two-hop neighbourhood	543.3 k	98.3 k	**5.53**×	yes
`read.incidence.parallel`	Incidence sets, 8 threads	36.56 M	2.25 M	**16.3**×	yes
`latency.read`	One lookup per read txn	3.83 M	457.0 k	**8.39**×	yes
`read.incidence.cold`	Incidence sets after reopen	4.04 M	95.9 k	**42.2**×	yes
`read.incidence.churned`	Incidence sets after deletes	8.40 M	2.10 M	**3.99**×	yes
`large.scan`	Scan the giant hyperedge	4.05 G	1.12 G	**3.61**×	yes
`large.probe`	Membership probe, giant hyperedge	17.21 M	1.82 M	**9.43**×	yes
`mixed.read`	Reads beside a 1,000 updates/s writer	2.46 M	1.13 M	**2.17**×	n/a
`reopen`	Clean reopen	111 ms	924 ms	**8.29**×	yes
`recover`	Open after SIGKILL	128 ms	76 ms	0.59×	n/a
`disk`	Disk, after load	92.4 MiB	346.4 MiB	**3.75**×	n/a
`disk.churned`	Disk, after deletes	117.1 MiB	219.8 MiB	**1.88**×	n/a
`disk.compacted`	Disk, after compaction	91.2 MiB	198.1 MiB	**2.17**×	n/a

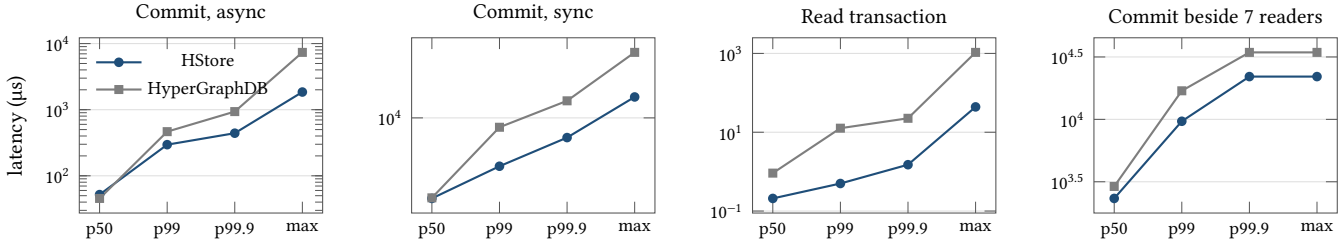


Figure 3: Latency percentiles, median of each percentile over runs (log scale). From left: single-update commits with asynchronous and with synchronous commits, single-lookup read transactions, and commits of the open-loop writer while 7 threads read.

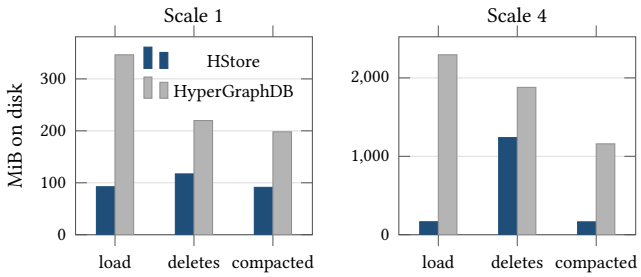


Figure 4: Bytes on disk after load, after deleting 20% of the hyperedges, and after a full space reclamation, median over runs.

are 117.1 MiB and 219.8 MiB. HStore's files grow at this point. The deletes write new pages, and the old ones stay on disk while any of the 64 retained generations can reach them, until a later checkpoint releases them and compaction removes them. After a full reclamation and a clean shutdown, HStore is back to 91.2 MiB, and HyperGraphDB is at 198.1 MiB.

4.6 Reopen and recovery

A clean close and reopen takes 111 ms in HStore and 924 ms in HyperGraphDB (8.29×). After the ingest process is killed with SIGKILL, opening the store takes 128 ms and 76 ms respectively, and the ranges do not overlap. HyperGraphDB opens a crashed store faster at scale 1. In every run of both systems, every node acknowledged before the kill was present after recovery.

4.7 Resources

HStore writes fewer bytes per operation than HyperGraphDB in every write workload of every configuration (fig. 5 and appendix A). A single-update commit writes 659 B in HStore and 6.9 KiB in HyperGraphDB. A batched update writes 224 B against 7.6 KiB. Over a whole run at scale 1, HStore wrote 98.1 MiB and HyperGraphDB 3.48 GiB.

Table 7 summarises memory and reclamation. HStore retained 189.8 MiB of heap after ingest at scale 1, against 118.4 MiB for HyperGraphDB. Per-workload allocation and

Table 6: S1 async: latency per operation, median of each percentile over runs.

		HStore				HyperGraphDB			
Workload	Operation	p50	p99	p99.9	max	p50	p99	p99.9	max
`latency.read`	Read transaction	0.2 µs	0.5 µs	1.5 µs	44 µs	0.9 µs	13 µs	23 µs	1.06 ms
`latency.commit`	Commit	52 µs	296 µs	441 µs	1.85 ms	45 µs	466 µs	936 µs	7.36 ms
`mixed.read`	Read beside writer	1.8 µs	10 µs	26 µs	28.51 ms	3.3 µs	55 µs	305 µs	28.50 ms
`mixed.write`	Commit beside readers	2.33 ms	9.65 ms	22.02 ms	22.02 ms	2.90 ms	16.94 ms	34.41 ms	34.41 ms
`compaction.baseline`	Commit, no compaction	3.31 ms	5.43 ms	8.80 ms	8.80 ms	3.36 ms	7.50 ms	10.52 ms	10.52 ms
`compaction`	Commit during compaction	3.36 ms	5.44 ms	6.17 ms	6.17 ms	3.24 ms	11.59 ms	84.26 ms	84.26 ms

Table 7: Resources per configuration, median over runs. HS: HStore, HG: HyperGraphDB. Space reclaimed is the drop in disk size across the compaction window; it is negative when the writer added more than the reclamation freed.

	S1 async		S1 sync		S4 async	
Measure	HS	HG	HS	HG	HS	HG
Retained heap after ingest	189.8 MiB	118.4 MiB	190.4 MiB	118.5 MiB	429.6 MiB	299.0 MiB
Bytes written, whole run	98.1 MiB	3.48 GiB	99.5 MiB	5.22 GiB	2.09 GiB	95.60 GiB
GC pause, whole run (ms)	197	565	226	773	2,421	28,839
Reclamation time (ms)	136	48	162	21	80	1,094,052
Space reclaimed	27.6 MiB	2.6 MiB	29.0 MiB	8.8 MiB	−3.1 MiB	720.1 MiB
Acknowledged before kill	25,000	25,000	25,000	25,000	100,000	100,000
Acknowledged lost (max)	0	0	0	0	0	0

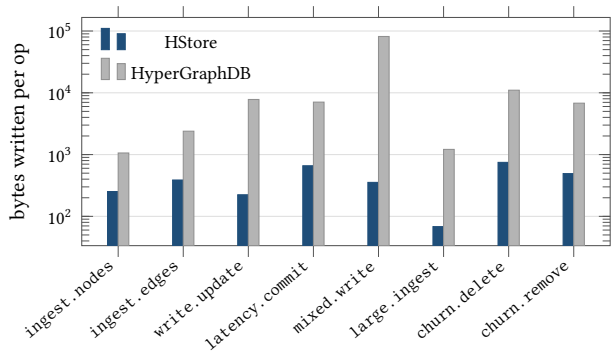


Figure 5: Bytes written per operation, S1 async, median over runs (log scale). Writes include each system's log, data files, checkpoints and cleaning.

GC time are in appendix A. HStore allocates less per operation than HyperGraphDB in every read workload of every configuration. At scale 1 it allocates more per inserted node and hyperedge, and more while recovering after a crash. At scale 4 it allocates less in those workloads as well.

4.8 Larger data: scale 4

At scale 4 the dataset has 1,453,427 incidences. After loading, HStore's files take 165.7 MiB and HyperGraphDB's 2.24 GiB, so both are larger than their 64 MiB caches, HyperGraphDB's by a much wider margin. HStore's leads hold or grow. Hyperedge insertion becomes 10.3× faster (21.1 k against 2.1 k per second), batched updates 33.1× and single-update commits 5.39×. Co-membership counts are 19.7× faster, incidence lookups 11.4×, and cold lookups after a reopen 88.9×.

Member enumeration is 3.22× faster: 5.58 M hyperedges per second against HyperGraphDB's 1.73 M. Two-hop neighbourhoods, which enumerate the members of every incident hyperedge, are 4.35× faster. During `read.members`, HStore read nothing from its files and allocated 0 B per lookup, so every catalog leaf the probes reached was already in its node cache. A cached catalog leaf holds its page bytes and only the records that have been read (section 2), which keeps the catalog within the 64 MiB budget. HyperGraphDB serves the same lookups from its atom cache, which holds each link's targets as a Java array and is not limited by the 64 MiB budget. It allocates 50 B per member lookup. A two-hop query allocates 1.2 KiB in HStore and 1.8 KiB in HyperGraphDB.

After the deletes, HStore's files grow to 1.21 GiB. The deletes rewrite a large part of the incidence index, and the replaced pages remain reachable from the 64 retained generations until the next restart releases that history. The benchmark restarts the store after the deletes, and during the 10 s `compaction.baseline` window that follows, HStore's background maintenance released and compacted that space. When the measured compaction window began, the explicit compaction took 80 ms and found almost nothing left to reclaim, and the final size was 164.5 MiB. JE's cleaner ran for 1,094,052 ms in the same window and brought HyperGraphDB from 1.84 GiB to 1.13 GiB.

A clean reopen is 5.61× faster in HStore, and opening after a crash takes 219 ms against 455 ms. Over the whole run, HStore wrote 2.09 GiB and HyperGraphDB 95.60 GiB, most of it during cleaning. HStore's single-update commits are faster at every percentile: 232 µs against 3.18 ms at p99, and 475 µs against 8.25 ms at p99.9. The median of the per-run maxima is also lower in HStore, 3.35 ms against 14.49 ms, but the slowest single commit across all runs was HStore's: 30.45 ms, against 18.84 ms for HyperGraphDB.

4.9 Synchronous commits

With synchronous commits, every acknowledged commit has been forced to stable storage. On this machine a forced write costs about 4 ms, and it dominates single-update transactions in both systems. HStore commits 258 of them per second and HyperGraphDB 236 (1.09×). The medians are 3.98 ms and 4.01 ms, one forced write each, and HStore's lead comes from the tail: 7.98 ms against 12.16 ms at p99.9.

Batched work spreads the forced write over 1,000 operations. HStore inserts nodes 1.81× as fast as HyperGraphDB and hyperedges 1.25× as fast (ranges overlapping), and updates values 7.34× as fast. The open-loop writer of `mixed` commits 10 updates every 10 ms. Both systems held that schedule in every run. Measured from their scheduled start, HStore's commits took 5.09 ms at the median and 19.11 ms at p99, and HyperGraphDB's 5.98 ms and 43.13 ms.

Under synchronous commits, JE reports 496.5 KiB written per single-update commit, against 6.9 KiB with asynchronous commits. The extra bytes come with the log flush that the adapter adds (section 2.3). We have not established what JE writes in that flush, so the bytes-written figures for synchronous single commits in appendix A should not be read as a comparison.

Reads do not commit, and HStore leads every read workload in this configuration as well at the median, with overlapping ranges for member enumeration and giant-edge scans (table 4). Recovery after SIGKILL takes 113 ms in HStore and 78 ms in HyperGraphDB at the median, and the ranges do not overlap. As in every other configuration, no run of either system lost an acknowledged commit.

5 Discussion

Incidence and intersection. An incidence lookup in HStore reads one entry of the reverse index. For most atoms the whole incidence set is stored inline in that entry, and for hubs it is the root of a small tree. The visitor API hands each hyperedge id to the caller without allocating, which is why `read.incidence` allocates about one byte per lookup. HyperGraphDB opens a JE cursor on a duplicate set, decodes 16-byte handles, and builds a result object per lookup. With a 64 MiB JE cache, part of each lookup also misses the cache and reads from the log files (appendix A). Co-membership adds a second effect. HStore intersects two sorted incidence trees and skips every subtree whose key range cannot match, so the cost follows the smaller set. HyperGraphDB plans `hg.and` as a general conjunctive query over two result sets. The two-hop workload combines both lookups, so it inherits both advantages.

Member enumeration. HyperGraphDB keeps a link's targets as a Java array in its atom cache, so enumerating the members of a cached link is an array loop with almost no allocation. HStore reads a hyperedge's members from its record in the catalog tree: from the leaf stored inline in the record for a small edge, or from the edge's own membership tree. What decides the result at scale 4 is how much heap a cached catalog leaf takes. A leaf holds about 80 records, each with its inline member list. Decoded in full, it takes roughly 11 times the bytes of its page image, and at scale 4 only about half of the catalog then fits in a 64 MiB node cache. An earlier HStore version (commit `459b8be`) decoded leaves in full, and its member enumeration at scale 4 ran at 0.13 times HyperGraphDB's rate, with most of the time spent decoding records that the lookup did not need. The version measured here frames each record by its length and decodes a record only when it is read, so a cached leaf costs its page bytes plus the records actually used. The catalog leaves that the lookups reach then stay in the cache, and member lookups read nothing from the files.

Tail latency. HStore readers pin an immutable generation and never wait for a writer, a checkpoint or a compaction. Writers wait only for the commit lock, which a checkpoint takes just to swap stored references into the head. HyperGraphDB reads run inside JE transactions, which take record locks, and JE's cleaner and checkpointer compete for the same latches and log. In 16 of the 18 latency measurements (six workloads in three configurations), the gap between the two systems is wider at p99 and p99.9 than at the median. The two exceptions are the `mixed` writer under synchronous commits and at scale 4. There the gap is widest at p99 and smaller at p99.9 than at the median, and since the writer makes 1,000 commits per run, its p99.9 is its slowest commit.

Giant hyperedges. HStore bulk-builds a large membership tree bottom-up in one pass, and a membership probe is one descent of that tree. HyperGraphDB stores the giant link as one record holding 50,000 handles (200,000 at scale 4). A membership probe goes through the node's incidence set instead.

Footprint and reclamation. Several layout choices keep HStore's files small: 8-byte ids stored as varint gaps instead of 16-byte handles, node images packed into 64-byte units, and small member trees and incidence sets stored inline in their parent record. HStore's log is also truncated at each checkpoint. A JE log keeps every superseded version until the cleaner copies the live entries out of a file and deletes it, and at scale 4 cleaning dominated HyperGraphDB's bytes written. HStore's weak point is the period right after a large delete. Pages replaced by the deletes stay on disk while any of the 64 retained generations can reach them, so the files grow first and shrink only after the history moves on, here at the restart that follows. An application that deletes heavily and needs its disk space back quickly would set a smaller `history_limit`.

Opening and recovery. After a clean shutdown, HStore reads one catalog image and finds nothing in the log to replay. HyperGraphDB's open takes longer and allocates far more heap: 1.72 GiB per reopen at scale 1, against 25.7 MiB for HStore. After a crash, HStore replays the logged key changes of every commit since its last checkpoint, and JE replays its log from its last checkpoint. HStore's replay cost

grows with the number of commits since the last checkpoint, which the write buffer and the checkpoint thresholds bound. At scale 1 HyperGraphDB opens the crashed store faster, 76 ms against 128 ms. HStore's time there is close to its clean reopen of the full dataset (111 ms), and it allocates 121.0 MiB while opening, against 78.0 MiB for HyperGraphDB. At scale 4 the order reverses: 219 ms for HStore against 455 ms.

What HStore spends. HStore retains more heap after ingest than HyperGraphDB (table 7). The retained heap includes nodes not yet written by a checkpoint, up to the 64 MiB write buffer, and the node cache, whose 64 MiB budget counts estimated rather than measured node sizes. At scale 1 it also allocates more per inserted node and hyperedge, opens a crashed store more slowly, and with asynchronous commits its median single-update commit is slightly slower than HyperGraphDB's. Its slowest single commit at scale 4 was slower than HyperGraphDB's, even though its p99 and p99.9 were lower.

6 Threats to validity

One machine, not idle. All numbers come from one laptop that was also running other work. Sampled before each run, the one-minute load average was between 3.4 and 18.3 (median 6.0) on 10 cores. That figure includes the tail of the previous run, whose parallel workloads use 8 threads, so only part of it is outside work. Absolute throughput will differ on a server, and short workloads are sensitive to interference. We report medians, show every run in appendix B, and mark ratios whose run ranges overlap. Conclusions in this report rest only on ratios that are large compared with that spread.

Durability. On this machine one `FileChannel.force` after a small write takes about 3.8 ms at the median, the cost of a write that reaches stable storage. Both systems pay it on every synchronous commit: HStore through its group commit, HyperGraphDB through the explicit log flush described in section 2.3. Without that flush, HyperGraphDB's synchronous configuration would not sync at all. The `recover` workload kills a process, not the machine, so it measures recovery time and the system's own buffering, not power-failure durability. HStore's behaviour under torn writes and lost syncs is covered by its crash-injection tests, which are outside the scope of this report.

Memory fits in RAM. At both scales the files fit in the operating system's page cache. Reads that miss a system's own 64 MiB cache are served from memory, not from the SSD. The cold-read numbers therefore measure decoding and cache admission, not device latency.

Cache budgets. HStore counts its 64 MiB node cache against an estimate of the heap each cached node takes, not a measured size. JE's 64 MiB budget is exact, but HyperGraphDB's atom and incidence caches are not capped in bytes. Neither side therefore holds exactly 64 MiB. The retained heap in table 7 shows what each kept after ingest.

Workload coverage. The dataset is synthetic. It uses set hyperedges without roles, properties or ordered members, and node values of a few bytes. Queries are single storage operations, not HQL or HyperGraphDB queries, so query planning is not measured. Workloads run in a fixed order within one process, so each one starts from the state the previous one left: cache contents, garbage, and for HStore, unwritten nodes waiting for a checkpoint.

Versions. HyperGraphDB 1.4 is built against Berkeley DB JE 5.0.73, which is older than current JE releases. We did not try other JE versions or tuning beyond the cache size. Both systems otherwise use their default settings.

Authorship. The HStore authors wrote the harness and both adapters. To limit bias, both adapters call the most direct storage API available, every read result is cross-checked between the systems, and both receive the same warm-up and cache budget. The harness, the adapters and the raw results are published with HStore.

7 Conclusion

At the storage level, HStore is faster than HyperGraphDB on the operations that hypergraph workloads repeat most: finding the hyperedges that contain a node, intersecting those sets, and updating values. It writes and stores far fewer bytes doing so. Its design gives a clear reason for each of these results. Keeping cached pages compact matters as much as the layout on disk: decoding only the catalog records that are read is what lets HStore's catalog stay within a 64 MiB cache at scale 4. HStore pays for its results with more retained heap, and at scale 1 it opens a crashed store more slowly than HyperGraphDB. Synchronous commits are bound by the device in both systems. Crash recovery lost no acknowledged commit in either system.

The numbers come from one laptop under load, and section 6 lists what that limits. The harness, the adapters, the raw results and the script that produced every table in this report are published with HStore, so each result can be checked or repeated on other hardware.

References

[1] B. Iordanov. HyperGraphDB: a generalized graph database. In *Web-Age Information Management (WAIM) Workshops*, LNCS 6185, pages 25–36. Springer, 2010.

[2] Oracle. *Berkeley DB Java Edition Architecture*. Oracle white paper, 2006.

[3] T. L. Veldhuizen. Leapfrog triejoin: a simple, worst-case optimal join algorithm. In *Proc. ICDT*, pages 96–106, 2014.

[4] H. Chu. MDB: a memory-mapped database and backend for OpenLDAP. In *Proc. LDAPCon*, 2011.

[5] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, P. Schwarz. ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM TODS* 17(1):94–162, 1992.

[6] M. Rosenblum, J. K. Ousterhout. The design and implementation of a log-structured file system. *ACM TOCS* 10(1):26–52, 1992.

A Full results

Each cell is the median over runs. Throughput is in operations per second (k = 10^3, M = 10^6, G = 10^9). Ratios above 1 favour HStore. "Agree" is *yes* when both systems returned the same checksum in every run.

A.1 Scale 1, asynchronous commits

Workload	Operation	HStore	HyperGraphDB	Ratio	Agree
`ingest.nodes`	Insert nodes (1,000 per txn)	67.7 k	37.7 k	**1.80**×	yes
`ingest.edges`	Insert hyperedges (1,000 per txn)	50.0 k	35.0 k	**1.43**×	yes
`write.update`	Update node values (1,000 per txn)	69.9 k	13.0 k	**5.38**×	yes
`latency.commit`	Single-update transactions	12.4 k	14.1 k	0.88×	yes
`churn.delete`	Delete hyperedges (1,000 per txn)	24.5 k	7.9 k	**3.10**×	yes
`churn.remove`	Remove one member (1,000 per txn)	43.2 k	17.5 k	**2.47**×	yes
`large.ingest`	Build one giant hyperedge	610.1 k	116.3 k	**5.25**×	yes
`read.incidence`	Incidence set of a node	6.26 M	1.49 M	**4.21**×	yes
`read.members`	Members of a hyperedge	6.11 M	2.66 M	**2.30**×	yes
`read.comembership`	Co-membership count of a pair	610.3 k	53.9 k	**11.3**×	yes
`read.twohop`	Two-hop neighbourhood	543.3 k	98.3 k	**5.53**×	yes
`read.incidence.parallel`	Incidence sets, 8 threads	36.56 M	2.25 M	**16.3**×	yes
`latency.read`	One lookup per read txn	3.83 M	457.0 k	**8.39**×	yes
`read.incidence.cold`	Incidence sets after reopen	4.04 M	95.9 k	**42.2**×	yes
`read.incidence.churned`	Incidence sets after deletes	8.40 M	2.10 M	**3.99**×	yes
`large.scan`	Scan the giant hyperedge	4.05 G	1.12 G	**3.61**×	yes
`large.probe`	Membership probe, giant hyperedge	17.21 M	1.82 M	**9.43**×	yes
`mixed.read`	Reads beside a 1,000 updates/s writer	2.46 M	1.13 M	**2.17**×	n/a
`reopen`	Clean reopen	111 ms	924 ms	**8.29**×	yes
`recover`	Open after SIGKILL	128 ms	76 ms	0.59×	n/a
`disk`	Disk, after load	92.4 MiB	346.4 MiB	**3.75**×	n/a
`disk.churned`	Disk, after deletes	117.1 MiB	219.8 MiB	**1.88**×	n/a
`disk.compacted`	Disk, after compaction	91.2 MiB	198.1 MiB	**2.17**×	n/a

Latency (median of each percentile over runs)

		HStore				HyperGraphDB			
Workload	Operation	p50	p99	p99.9	max	p50	p99	p99.9	max
`latency.read`	Read transaction	0.2 µs	0.5 µs	1.5 µs	44 µs	0.9 µs	13 µs	23 µs	1.06 ms
`latency.commit`	Commit	52 µs	296 µs	441 µs	1.85 ms	45 µs	466 µs	936 µs	7.36 ms
`mixed.read`	Read beside writer	1.8 µs	10 µs	26 µs	28.51 ms	3.3 µs	55 µs	305 µs	28.50 ms
`mixed.write`	Commit beside readers	2.33 ms	9.65 ms	22.02 ms	22.02 ms	2.90 ms	16.94 ms	34.41 ms	34.41 ms
`compaction.baseline`	Commit, no compaction	3.31 ms	5.43 ms	8.80 ms	8.80 ms	3.36 ms	7.50 ms	10.52 ms	10.52 ms
`compaction`	Commit during compaction	3.36 ms	5.44 ms	6.17 ms	6.17 ms	3.24 ms	11.59 ms	84.26 ms	84.26 ms

	Bytes written per operation		Write amplification		
Workload	HStore	HyperGraphDB	HStore	HyperGraphDB	Ratio
`ingest.nodes`	252 B	1.0 KiB	14.2×	59.7×	**4.21**×
`ingest.edges`	388 B	2.3 KiB	10.5×	64.6×	**6.17**×
`write.update`	224 B	7.6 KiB	7.0×	243.6×	**34.9**×
`latency.commit`	659 B	6.9 KiB	20.6×	220.7×	**10.7**×
`mixed.write`	355 B	79.6 KiB	11.1×	2548.6×	**230**×
`large.ingest`	68 B	1.2 KiB	8.5×	151.6×	**17.8**×
`churn.delete`	748 B	10.8 KiB	20.2×	297.9×	**14.7**×
`churn.remove`	492 B	6.6 KiB	30.7×	424.4×	**13.8**×

	Heap allocated per operation		GC pause (ms)	
Workload	HStore	HyperGraphDB	HStore	HyperGraphDB
`ingest.nodes`	19.1 KiB	18.2 KiB	0	0
`ingest.edges`	36.3 KiB	29.3 KiB	54	57
`write.update`	23.2 KiB	168.9 KiB	6	24
`latency.commit`	76.9 KiB	142.2 KiB	0	6
`churn.delete`	53.2 KiB	262.2 KiB	55	38
`churn.remove`	26.1 KiB	137.7 KiB	32	14
`large.ingest`	2.3 KiB	16.0 KiB	0	9
`read.incidence`	1 B	98 B	0	0
`read.members`	40 B	49 B	0	0
`read.comembership`	4.0 KiB	7.8 KiB	0	40
`read.twohop`	373 B	1.8 KiB	0	0
`read.incidence.parallel`	2 B	929 B	0	4
`latency.read`	269 B	4.6 KiB	0	0
`read.incidence.cold`	280 B	5.6 KiB	0	41
`read.incidence.churned`	1 B	97 B	0	0
`large.scan`	0 B	0 B	0	0
`large.probe`	0 B	98 B	0	0
`mixed.read`	98 B	2.5 KiB	19	152
`reopen`	25.7 MiB	1.72 GiB	0	8
`recover`	121.0 MiB	78.0 MiB	0	0

A.2 Scale 1, synchronous commits

Workload	Operation	HStore	HyperGraphDB	Ratio	Agree
`ingest.nodes`	Insert nodes (1,000 per txn)	55.8 k	30.9 k	**1.81**×	yes
`ingest.edges`	Insert hyperedges (1,000 per txn)	37.1 k	29.6 k	**1.25**×	yes
`write.update`	Update node values (1,000 per txn)	64.1 k	8.7 k	**7.34**×	yes
`latency.commit`	Single-update transactions	258	236	**1.09**×	yes
`churn.delete`	Delete hyperedges (1,000 per txn)	26.9 k	5.5 k	**4.91**×	yes
`churn.remove`	Remove one member (1,000 per txn)	36.8 k	11.9 k	**3.10**×	yes
`large.ingest`	Build one giant hyperedge	611.6 k	68.5 k	**8.93**×	yes
`read.incidence`	Incidence set of a node	6.91 M	997.2 k	**6.93**×	yes
`read.members`	Members of a hyperedge	5.42 M	2.38 M	**2.28**×	yes
`read.comembership`	Co-membership count of a pair	623.4 k	40.5 k	**15.4**×	yes
`read.twohop`	Two-hop neighbourhood	349.1 k	104.6 k	**3.34**×	yes
`read.incidence.parallel`	Incidence sets, 8 threads	26.82 M	1.96 M	**13.7**×	yes
`latency.read`	One lookup per read txn	3.59 M	437.8 k	**8.21**×	yes
`read.incidence.cold`	Incidence sets after reopen	2.93 M	104.7 k	**28.0**×	yes
`read.incidence.churned`	Incidence sets after deletes	8.44 M	1.13 M	**7.47**×	yes
`large.scan`	Scan the giant hyperedge	3.86 G	921.64 M	**4.18**×	yes
`large.probe`	Membership probe, giant hyperedge	11.86 M	1.23 M	**9.65**×	yes
`mixed.read`	Reads beside a 1,000 updates/s writer	2.28 M	1.05 M	**2.18**×	n/a
`reopen`	Clean reopen	128 ms	1,077 ms	**8.44**×	yes
`recover`	Open after SIGKILL	113 ms	78 ms	0.69×	n/a
`disk`	Disk, after load	93.3 MiB	321.7 MiB	**3.45**×	n/a
`disk.churned`	Disk, after deletes	118.5 MiB	256.5 MiB	**2.17**×	n/a
`disk.compacted`	Disk, after compaction	91.2 MiB	190.4 MiB	**2.09**×	n/a

Latency (median of each percentile over runs)

		HStore				HyperGraphDB			
Workload	Operation	p50	p99	p99.9	max	p50	p99	p99.9	max
`latency.read`	Read transaction	0.2 µs	0.5 µs	2.9 µs	39 µs	1.0 µs	14 µs	24 µs	145 µs
`latency.commit`	Commit	3.98 ms	5.75 ms	7.98 ms	12.70 ms	4.01 ms	8.98 ms	12.16 ms	21.16 ms
`mixed.read`	Read beside writer	1.9 µs	12 µs	28 µs	38.67 ms	3.8 µs	65 µs	397 µs	25.44 ms
`mixed.write`	Commit beside readers	5.09 ms	19.11 ms	47.13 ms	47.13 ms	5.98 ms	43.13 ms	52.94 ms	52.94 ms
`compaction.baseline`	Commit, no compaction	6.33 ms	8.58 ms	11.08 ms	11.08 ms	6.24 ms	12.84 ms	22.48 ms	22.48 ms
`compaction`	Commit during compaction	6.39 ms	9.02 ms	11.12 ms	11.12 ms	6.40 ms	29.75 ms	93.96 ms	93.96 ms

	Bytes written per operation		Write amplification		
Workload	HStore	HyperGraphDB	HStore	HyperGraphDB	Ratio
`ingest.nodes`	252 B	1.0 KiB	14.2×	59.8×	**4.22**×
`ingest.edges`	397 B	2.3 KiB	10.7×	64.7×	**6.04**×
`write.update`	224 B	9.5 KiB	7.0×	305.1×	**43.7**×
`latency.commit`	659 B	496.5 KiB	20.6×	15889.6×	**771**×
`mixed.write`	355 B	69.2 KiB	11.1×	2215.3×	**200**×
`large.ingest`	68 B	1.5 KiB	8.5×	197.5×	**23.2**×
`churn.delete`	746 B	10.2 KiB	20.2×	282.2×	**14.0**×
`churn.remove`	496 B	6.8 KiB	31.0×	432.3×	**13.9**×

	Heap allocated per operation		GC pause (ms)	
Workload	HStore	HyperGraphDB	HStore	HyperGraphDB
`ingest.nodes`	18.9 KiB	18.2 KiB	0	0
`ingest.edges`	35.8 KiB	29.5 KiB	59	70
`write.update`	22.9 KiB	210.3 KiB	0	25
`latency.commit`	129.7 KiB	8.6 MiB	6	192
`churn.delete`	55.0 KiB	232.9 KiB	35	50
`churn.remove`	30.4 KiB	143.8 KiB	36	15
`large.ingest`	2.3 KiB	24.4 KiB	0	8
`read.incidence`	1 B	99 B	0	0
`read.members`	40 B	49 B	0	0
`read.comembership`	4.0 KiB	7.8 KiB	0	58
`read.twohop`	373 B	1.8 KiB	0	0
`read.incidence.parallel`	2 B	920 B	0	5
`latency.read`	229 B	5.3 KiB	0	0
`read.incidence.cold`	281 B	5.6 KiB	0	15
`read.incidence.churned`	1 B	98 B	0	0
`large.scan`	0 B	0 B	0	0
`large.probe`	64 B	98 B	0	0
`mixed.read`	106 B	2.3 KiB	33	139
`reopen`	25.7 MiB	1.73 GiB	0	8
`recover`	121.5 MiB	78.2 MiB	0	0

A.3 Scale 4, asynchronous commits

Workload	Operation	HStore	HyperGraphDB	Ratio	Agree
`ingest.nodes`	Insert nodes (1,000 per txn)	40.0 k	28.7 k	**1.39**×	yes
`ingest.edges`	Insert hyperedges (1,000 per txn)	21.1 k	2.1 k	**10.3**×	yes
`write.update`	Update node values (1,000 per txn)	51.9 k	1.6 k	**33.1**×	yes
`latency.commit`	Single-update transactions	12.2 k	2.3 k	**5.39**×	yes
`churn.delete`	Delete hyperedges (1,000 per txn)	3.4 k	1.4 k	**2.38**×	yes
`churn.remove`	Remove one member (1,000 per txn)	5.8 k	1.9 k	**3.01**×	yes
`large.ingest`	Build one giant hyperedge	489.3 k	12.1 k	**40.5**×	yes
`read.incidence`	Incidence set of a node	6.57 M	574.9 k	**11.4**×	yes
`read.members`	Members of a hyperedge	5.58 M	1.73 M	**3.22**×	yes
`read.comembership`	Co-membership count of a pair	357.9 k	18.1 k	**19.7**×	yes
`read.twohop`	Two-hop neighbourhood	311.3 k	71.6 k	**4.35**×	yes
`read.incidence.parallel`	Incidence sets, 8 threads	14.85 M	1.46 M	**10.2**×	yes
`latency.read`	One lookup per read txn	2.36 M	350.7 k	**6.73**×	yes
`read.incidence.cold`	Incidence sets after reopen	1.16 M	13.1 k	**88.9**×	yes
`read.incidence.churned`	Incidence sets after deletes	5.68 M	1.02 M	**5.59**×	yes
`large.scan`	Scan the giant hyperedge	3.64 G	431.47 M	**8.43**×	yes
`large.probe`	Membership probe, giant hyperedge	11.71 M	707.0 k	**16.6**×	yes
`mixed.read`	Reads beside a 1,000 updates/s writer	1.98 M	855.9 k	**2.32**×	n/a
`reopen`	Clean reopen	206 ms	1,157 ms	**5.61**×	yes
`recover`	Open after SIGKILL	219 ms	455 ms	**2.08**×	n/a
`disk`	Disk, after load	165.7 MiB	2.24 GiB	**13.9**×	n/a
`disk.churned`	Disk, after deletes	1.21 GiB	1.84 GiB	**1.52**×	n/a
`disk.compacted`	Disk, after compaction	164.5 MiB	1.13 GiB	**7.05**×	n/a

Latency (median of each percentile over runs)

		HStore				HyperGraphDB			
Workload	Operation	p50	p99	p99.9	max	p50	p99	p99.9	max
`latency.read`	Read transaction	0.4 µs	0.6 µs	1.5 µs	62 µs	1.3 µs	22 µs	35 µs	6.30 ms
`latency.commit`	Commit	55 µs	232 µs	475 µs	3.35 ms	273 µs	3.18 ms	8.25 ms	14.49 ms
`mixed.read`	Read beside writer	1.8 µs	13 µs	68 µs	50.18 ms	2.5 µs	45 µs	968 µs	42.13 ms
`mixed.write`	Commit beside readers	3.07 ms	23.53 ms	58.25 ms	58.25 ms	6.16 ms	74.75 ms	99.38 ms	99.38 ms
`compaction.baseline`	Commit, no compaction	5.13 ms	9.80 ms	10.77 ms	10.77 ms	3.96 ms	17.74 ms	41.02 ms	41.02 ms
`compaction`	Commit during compaction	4.78 ms	9.95 ms	21.66 ms	21.66 ms	4.88 ms	64.29 ms	181.83 ms	393.39 ms

	Bytes written per operation		Write amplification		
Workload	HStore	HyperGraphDB	HStore	HyperGraphDB	Ratio
`ingest.nodes`	315 B	1.5 KiB	17.1×	86.0×	**5.03**×
`ingest.edges`	1023 B	28.2 KiB	27.6×	778.3×	**28.2**×
`write.update`	508 B	38.9 KiB	15.9×	1244.8×	**78.4**×
`latency.commit`	1.7 KiB	26.2 KiB	55.5×	837.8×	**15.1**×
`mixed.write`	5.5 KiB	48.7 KiB	177.6×	1560.0×	**8.78**×
`large.ingest`	73 B	6.1 KiB	9.1×	784.1×	**86.1**×
`churn.delete`	16.5 KiB	41.0 KiB	453.9×	1129.9×	**2.49**×
`churn.remove`	14.5 KiB	22.7 KiB	928.2×	1455.1×	**1.57**×

	Heap allocated per operation		GC pause (ms)	
Workload	HStore	HyperGraphDB	HStore	HyperGraphDB
`ingest.nodes`	25.6 KiB	25.8 KiB	171	122
`ingest.edges`	101.6 KiB	871.1 KiB	1001	2088
`write.update`	50.9 KiB	1.2 MiB	33	132
`latency.commit`	139.2 KiB	819.8 KiB	0	16
`churn.delete`	434.2 KiB	1.3 MiB	693	804
`churn.remove`	194.7 KiB	746.9 KiB	162	243
`large.ingest`	4.1 KiB	172.6 KiB	25	179
`read.incidence`	1 B	99 B	0	0
`read.members`	0 B	50 B	0	0
`read.comembership`	4.0 KiB	8.2 KiB	0	23
`read.twohop`	1.2 KiB	1.8 KiB	0	0
`read.incidence.parallel`	131 B	1.0 KiB	0	13
`latency.read`	293 B	7.0 KiB	0	6
`read.incidence.cold`	595 B	23.9 KiB	0	117
`read.incidence.churned`	1 B	98 B	0	0
`large.scan`	0 B	0 B	0	0
`large.probe`	0 B	100 B	0	0
`mixed.read`	118 B	2.9 KiB	79	176
`reopen`	61.7 MiB	1.97 GiB	0	13
`recover`	249.0 MiB	323.2 MiB	0	0

B Run-to-run variability

The tables below give the minimum and maximum over runs and the coefficient of variation (standard deviation over mean) of every throughput workload, followed by the value of every run.

B.1 Scale 1, asynchronous commits

	HStore		HyperGraphDB	
Workload	min–max	CV	min–max	CV
`ingest.nodes`	51.1 k–73.0 k	12%	32.8 k–40.0 k	7%
`ingest.edges`	42.3 k–54.8 k	8%	33.3 k–39.5 k	6%
`write.update`	50.3 k–108.0 k	26%	10.6 k–19.1 k	21%
`latency.commit`	10.4 k–14.9 k	12%	7.2 k–24.6 k	38%
`churn.delete`	20.2 k–29.9 k	14%	5.9 k–9.1 k	14%
`churn.remove`	39.0 k–49.7 k	9%	16.1 k–19.8 k	8%
`large.ingest`	286.7 k–755.1 k	29%	83.9 k–136.1 k	16%
`read.incidence`	3.84 M–8.19 M	23%	1.40 M–1.59 M	4%
`read.members`	2.15 M–6.41 M	31%	1.62 M–3.49 M	24%
`read.comembership`	560.3 k–788.4 k	14%	2.2 k–76.6 k	79%
`read.twohop`	335.8 k–574.0 k	21%	71.0 k–148.8 k	25%
`read.incidence.parallel`	20.95 M–40.59 M	21%	1.95 M–2.68 M	10%
`latency.read`	2.41 M–4.72 M	26%	372.3 k–558.3 k	15%
`read.incidence.cold`	3.46 M–4.14 M	8%	89.0 k–137.0 k	17%
`read.incidence.churned`	7.85 M–9.26 M	6%	932.1 k–2.52 M	29%
`large.scan`	3.51 G–4.91 G	12%	590.30 M–1.30 G	25%
`large.probe`	9.63 M–19.64 M	21%	865.6 k–2.06 M	35%
`mixed.read`	2.37 M–2.54 M	2%	1.06 M–1.20 M	4%

Workload	Engine	Run 1	Run 2	Run 3	Run 4	Run 5
`ingest.nodes`	HStore	51.1 k	69.5 k	67.7 k	73.0 k	61.8 k
	HyperGraphDB	40.0 k	39.8 k	32.8 k	37.7 k	35.7 k
`ingest.edges`	HStore	42.3 k	54.8 k	50.0 k	48.1 k	51.1 k
	HyperGraphDB	35.0 k	33.3 k	39.5 k	37.3 k	34.2 k
`write.update`	HStore	65.4 k	74.3 k	50.3 k	69.9 k	108.0 k
	HyperGraphDB	19.1 k	13.4 k	10.6 k	12.1 k	13.0 k
`latency.commit`	HStore	12.4 k	11.6 k	10.4 k	12.8 k	14.9 k
	HyperGraphDB	24.6 k	13.5 k	14.1 k	7.2 k	14.8 k
`churn.delete`	HStore	20.2 k	24.5 k	22.4 k	28.2 k	29.9 k
	HyperGraphDB	9.1 k	8.9 k	7.9 k	5.9 k	7.7 k
`churn.remove`	HStore	43.2 k	47.9 k	42.7 k	49.7 k	39.0 k
	HyperGraphDB	16.1 k	17.5 k	19.8 k	16.2 k	18.3 k
`large.ingest`	HStore	286.7 k	486.0 k	610.1 k	649.2 k	755.1 k
	HyperGraphDB	104.5 k	136.1 k	125.0 k	83.9 k	116.3 k
`read.incidence`	HStore	6.26 M	6.54 M	5.49 M	8.19 M	3.84 M
	HyperGraphDB	1.56 M	1.59 M	1.40 M	1.48 M	1.49 M
`read.members`	HStore	2.15 M	5.13 M	6.31 M	6.11 M	6.41 M
	HyperGraphDB	3.00 M	2.66 M	1.62 M	2.40 M	3.49 M
`read.comembership`	HStore	610.3 k	605.4 k	788.4 k	764.4 k	560.3 k
	HyperGraphDB	2.2 k	76.6 k	63.0 k	53.9 k	2.4 k
`read.twohop`	HStore	369.2 k	543.3 k	574.0 k	335.8 k	561.3 k
	HyperGraphDB	71.0 k	148.8 k	126.4 k	91.5 k	98.3 k
`read.incidence.parallel`	HStore	29.79 M	40.59 M	36.58 M	20.95 M	36.56 M
	HyperGraphDB	2.33 M	2.68 M	2.25 M	1.95 M	2.18 M
`latency.read`	HStore	3.83 M	4.36 M	2.41 M	2.57 M	4.72 M
	HyperGraphDB	432.0 k	457.0 k	551.9 k	372.3 k	558.3 k
`read.incidence.cold`	HStore	3.46 M	4.14 M	4.04 M	3.48 M	4.14 M
	HyperGraphDB	90.6 k	89.0 k	137.0 k	116.0 k	95.9 k
`read.incidence.churned`	HStore	9.26 M	8.40 M	8.27 M	7.85 M	8.81 M
	HyperGraphDB	932.1 k	2.52 M	1.80 M	2.38 M	2.10 M

Workload	Engine	Run 1	Run 2	Run 3	Run 4	Run 5
large.scan	HStore	4.41 G	4.05 G	3.51 G	3.70 G	4.91 G
	HyperGraphDB	1.30 G	986.65 M	1.28 G	1.12 G	590.30 M
large.probe	HStore	19.64 M	17.21 M	17.95 M	16.89 M	9.63 M
	HyperGraphDB	2.06 M	865.6 k	1.82 M	877.6 k	1.94 M
mixed.read	HStore	2.54 M	2.47 M	2.42 M	2.46 M	2.37 M
	HyperGraphDB	1.13 M	1.20 M	1.06 M	1.18 M	1.13 M
reopen	HStore	111 ms	107 ms	112 ms	111 ms	131 ms
	HyperGraphDB	1361 ms	752 ms	976 ms	674 ms	924 ms
recover	HStore	128 ms	153 ms	129 ms	106 ms	106 ms
	HyperGraphDB	73 ms	61 ms	79 ms	88 ms	76 ms
disk	HStore	92.4 MiB	90.4 MiB	92.6 MiB	91.9 MiB	92.6 MiB
	HyperGraphDB	346.4 MiB	371.8 MiB	311.1 MiB	361.3 MiB	289.6 MiB
disk.churned	HStore	114.1 MiB	115.7 MiB	117.9 MiB	117.1 MiB	117.7 MiB
	HyperGraphDB	283.5 MiB	213.5 MiB	302.1 MiB	219.8 MiB	203.2 MiB
disk.compacted	HStore	91.2 MiB	91.2 MiB	91.2 MiB	91.2 MiB	91.2 MiB
	HyperGraphDB	251.7 MiB	198.1 MiB	265.7 MiB	196.1 MiB	188.1 MiB

B.2 Scale 1, synchronous commits

	HStore		HyperGraphDB	
Workload	min–max	CV	min–max	CV
ingest.nodes	25.6 k–60.8 k	28%	25.2 k–33.0 k	9%
ingest.edges	16.9 k–43.5 k	30%	13.7 k–32.1 k	27%
write.update	42.5 k–77.5 k	20%	3.6 k–10.6 k	35%
latency.commit	244–260	2%	229–237	2%
churn.delete	10.0 k–28.1 k	32%	4.2 k–8.3 k	24%
churn.remove	21.2 k–40.9 k	25%	9.2 k–14.2 k	15%
large.ingest	330.7 k–701.3 k	26%	44.2 k–124.9 k	37%
read.incidence	2.18 M–8.49 M	35%	455.0 k–1.40 M	32%
read.members	2.35 M–6.91 M	40%	1.14 M–3.35 M	36%
read.comembership	226.0 k–750.3 k	33%	28.2 k–76.5 k	36%
read.twohop	212.3 k–575.3 k	37%	68.9 k–118.5 k	17%
read.incidence.parallel	8.34 M–41.00 M	49%	1.35 M–2.04 M	15%
latency.read	1.62 M–4.60 M	38%	274.3 k–695.1 k	31%
read.incidence.cold	2.17 M–4.18 M	28%	94.9 k–142.6 k	15%
read.incidence.churned	6.58 M–9.03 M	12%	982.4 k–1.80 M	24%
large.scan	1.07 G–4.21 G	35%	673.18 M–1.08 G	16%
large.probe	8.96 M–17.24 M	22%	819.9 k–1.81 M	29%
mixed.read	1.94 M–2.35 M	7%	885.1 k–1.09 M	8%

Workload	Engine	Run 1	Run 2	Run 3	Run 4	Run 5
ingest.nodes	HStore	56.2 k	60.8 k	55.8 k	25.6 k	37.7 k
	HyperGraphDB	31.0 k	30.5 k	25.2 k	33.0 k	30.9 k
ingest.edges	HStore	39.5 k	43.5 k	37.1 k	26.1 k	16.9 k
	HyperGraphDB	30.4 k	32.1 k	29.6 k	21.9 k	13.7 k
write.update	HStore	77.5 k	64.1 k	68.8 k	53.5 k	42.5 k
	HyperGraphDB	10.6 k	8.7 k	9.7 k	5.4 k	3.6 k
latency.commit	HStore	258	260	253	258	244
	HyperGraphDB	237	237	236	229	229
churn.delete	HStore	26.9 k	28.1 k	27.2 k	17.4 k	10.0 k
	HyperGraphDB	7.4 k	5.5 k	8.3 k	4.2 k	5.4 k
churn.remove	HStore	40.9 k	38.8 k	36.8 k	23.7 k	21.2 k
	HyperGraphDB	14.2 k	13.1 k	9.2 k	10.6 k	11.9 k
large.ingest	HStore	701.3 k	668.1 k	611.6 k	433.3 k	330.7 k
	HyperGraphDB	108.8 k	68.5 k	124.9 k	44.2 k	62.6 k
read.incidence	HStore	6.20 M	8.49 M	2.18 M	6.91 M	7.93 M
	HyperGraphDB	847.2 k	997.2 k	1.40 M	1.03 M	455.0 k

Workload	Engine	Run 1	Run 2	Run 3	Run 4	Run 5
read.members	HStore	5.42 M	6.04 M	6.91 M	2.60 M	2.35 M
	HyperGraphDB	2.53 M	3.35 M	2.38 M	1.54 M	1.14 M
read.comembership	HStore	750.3 k	623.4 k	728.5 k	557.4 k	226.0 k
	HyperGraphDB	45.4 k	76.5 k	40.5 k	28.2 k	36.5 k
read.twohop	HStore	569.1 k	286.3 k	575.3 k	349.1 k	212.3 k
	HyperGraphDB	118.5 k	105.1 k	104.6 k	104.2 k	68.9 k
read.incidence.parallel	HStore	41.00 M	26.82 M	8.34 M	33.89 M	14.27 M
	HyperGraphDB	2.04 M	1.96 M	2.01 M	1.35 M	1.56 M
latency.read	HStore	4.60 M	3.59 M	4.30 M	1.89 M	1.62 M
	HyperGraphDB	695.1 k	437.8 k	554.8 k	368.8 k	274.3 k
read.incidence.cold	HStore	4.13 M	4.18 M	2.17 M	2.93 M	2.24 M
	HyperGraphDB	142.6 k	109.2 k	104.7 k	94.9 k	103.9 k
read.incidence.churned	HStore	8.68 M	6.58 M	9.03 M	8.44 M	7.10 M
	HyperGraphDB	1.16 M	1.80 M	982.4 k	1.13 M	1.02 M
large.scan	HStore	4.01 G	3.86 G	3.22 G	4.21 G	1.07 G
	HyperGraphDB	921.64 M	1.08 G	922.69 M	790.04 M	673.18 M
large.probe	HStore	17.24 M	11.13 M	14.43 M	11.86 M	8.96 M
	HyperGraphDB	1.27 M	1.23 M	1.81 M	819.9 k	900.8 k
mixed.read	HStore	2.35 M	2.29 M	2.28 M	2.07 M	1.94 M
	HyperGraphDB	1.05 M	1.07 M	1.09 M	908.9 k	885.1 k
reopen	HStore	128 ms	119 ms	122 ms	144 ms	148 ms
	HyperGraphDB	858 ms	1158 ms	1245 ms	1077 ms	912 ms
recover	HStore	111 ms	184 ms	113 ms	107 ms	118 ms
	HyperGraphDB	78 ms	76 ms	78 ms	78 ms	54 ms
disk	HStore	93.3 MiB	93.7 MiB	91.8 MiB	92.1 MiB	93.6 MiB
	HyperGraphDB	305.6 MiB	317.5 MiB	350.8 MiB	321.7 MiB	337.1 MiB
disk.churned	HStore	118.5 MiB	118.9 MiB	117.1 MiB	117.7 MiB	127.5 MiB
	HyperGraphDB	199.4 MiB	195.4 MiB	271.5 MiB	256.5 MiB	276.1 MiB
disk.compacted	HStore	91.2 MiB	91.2 MiB	91.2 MiB	91.2 MiB	91.2 MiB
	HyperGraphDB	189.9 MiB	182.5 MiB	277.1 MiB	190.4 MiB	269.3 MiB

B.3 Scale 4, asynchronous commits

	HStore		HyperGraphDB	
Workload	min–max	CV	min–max	CV
ingest.nodes	34.0 k–57.6 k	23%	26.7 k–38.9 k	17%
ingest.edges	20.0 k–27.3 k	14%	1.7 k–2.7 k	19%
write.update	22.0 k–67.6 k	40%	1.4 k–2.3 k	21%
latency.commit	8.4 k–16.8 k	28%	1.8 k–2.5 k	14%
churn.delete	2.2 k–4.3 k	26%	1.3 k–1.5 k	5%
churn.remove	3.5 k–6.0 k	22%	1.7 k–2.7 k	21%
large.ingest	357.7 k–530.4 k	16%	12.0 k–12.3 k	1%
read.incidence	6.37 M–7.07 M	4%	549.3 k–1.29 M	43%
read.members	4.56 M–5.93 M	11%	1.45 M–1.83 M	10%
read.comembership	306.4 k–380.5 k	9%	17.3 k–53.8 k	57%
read.twohop	226.9 k–397.0 k	22%	46.8 k–96.7 k	28%
read.incidence.parallel	9.71 M–17.93 M	24%	1.36 M–1.84 M	13%
latency.read	1.51 M–2.53 M	21%	205.1 k–361.6 k	23%
read.incidence.cold	984.6 k–2.01 M	32%	12.5 k–13.3 k	3%
read.incidence.churned	3.47 M–6.31 M	24%	531.4 k–1.30 M	34%
large.scan	2.47 G–3.73 G	17%	151.01 M–452.91 M	40%
large.probe	6.81 M–15.44 M	31%	363.1 k–1.12 M	42%
mixed.read	1.93 M–2.18 M	5%	840.0 k–1.01 M	8%

Workload	Engine	Run 1	Run 2	Run 3
ingest.nodes	HStore	40.0 k	57.6 k	34.0 k
	HyperGraphDB	28.7 k	38.9 k	26.7 k

Workload	Engine	Run 1	Run 2	Run 3
`ingest.edges`	HStore	21.1 k	27.3 k	20.0 k
	HyperGraphDB	2.1 k	2.7 k	1.7 k
`write.update`	HStore	51.9 k	67.6 k	22.0 k
	HyperGraphDB	1.6 k	2.3 k	1.4 k
`latency.commit`	HStore	12.2 k	16.8 k	8.4 k
	HyperGraphDB	1.8 k	2.5 k	2.3 k
`churn.delete`	HStore	3.4 k	4.3 k	2.2 k
	HyperGraphDB	1.5 k	1.4 k	1.3 k
`churn.remove`	HStore	5.8 k	6.0 k	3.5 k
	HyperGraphDB	2.7 k	1.9 k	1.7 k
`large.ingest`	HStore	489.3 k	530.4 k	357.7 k
	HyperGraphDB	12.1 k	12.3 k	12.0 k
`read.incidence`	HStore	7.07 M	6.57 M	6.37 M
	HyperGraphDB	574.9 k	1.29 M	549.3 k
`read.members`	HStore	4.56 M	5.58 M	5.93 M
	HyperGraphDB	1.73 M	1.45 M	1.83 M
`read.comembership`	HStore	306.4 k	380.5 k	357.9 k
	HyperGraphDB	18.1 k	53.8 k	17.3 k
`read.twohop`	HStore	226.9 k	397.0 k	311.3 k
	HyperGraphDB	71.6 k	96.7 k	46.8 k
`read.incidence.parallel`	HStore	9.71 M	14.85 M	17.93 M
	HyperGraphDB	1.36 M	1.84 M	1.46 M
`latency.read`	HStore	2.53 M	2.36 M	1.51 M
	HyperGraphDB	350.7 k	361.6 k	205.1 k
`read.incidence.cold`	HStore	1.16 M	2.01 M	984.6 k
	HyperGraphDB	13.1 k	13.3 k	12.5 k
`read.incidence.churned`	HStore	5.68 M	6.31 M	3.47 M
	HyperGraphDB	1.30 M	1.02 M	531.4 k
`large.scan`	HStore	3.64 G	3.73 G	2.47 G
	HyperGraphDB	151.01 M	452.91 M	431.47 M
`large.probe`	HStore	11.71 M	15.44 M	6.81 M
	HyperGraphDB	1.12 M	363.1 k	707.0 k
`mixed.read`	HStore	1.93 M	1.98 M	2.18 M
	HyperGraphDB	840.0 k	1.01 M	855.9 k
`reopen`	HStore	206 ms	130 ms	242 ms
	HyperGraphDB	1157 ms	1028 ms	1270 ms
`recover`	HStore	189 ms	219 ms	257 ms
	HyperGraphDB	455 ms	527 ms	327 ms
`disk`	HStore	165.7 MiB	164.9 MiB	166.4 MiB
	HyperGraphDB	2.31 GiB	2.24 GiB	2.16 GiB
`disk.churned`	HStore	1.21 GiB	1.20 GiB	1.21 GiB
	HyperGraphDB	1.80 GiB	1.92 GiB	1.84 GiB
`disk.compacted`	HStore	164.5 MiB	164.5 MiB	164.5 MiB
	HyperGraphDB	1.13 GiB	1.15 GiB	1.09 GiB

C Reproducing the results

The harness is the `benchmarks` module of the HStore repository. HyperGraphDB and JE are built from source at the commits named in table 1.

```
benchmarks/install-hypergraphdb.sh      # builds HyperGraphDB 99485a1 and JE 5.0.73
SCALE=1 DURABILITY=async CACHE_MB=64 RUNS=5 THREADS=8 benchmarks/run.sh
SCALE=1 DURABILITY=sync  CACHE_MB=64 RUNS=5 THREADS=8 benchmarks/run.sh
SCALE=4 DURABILITY=async CACHE_MB=64 RUNS=3 THREADS=8 benchmarks/run.sh
```

`run.sh` builds the module, then runs each system `RUNS` times in alternation, one JVM per run, with these flags:

```
java -Xms4g -Xmx4g -XX:+UseParallelGC \
     --add-opens java.base/java.lang=ALL-UNNAMED \
     --add-opens java.base/java.lang.ref=ALL-UNNAMED \
     --add-opens java.base/java.util=ALL-UNNAMED \
     io.hstore.bench.Comparison run --store <hstore|hypergraphdb> \
     --scale <s> --durability <async|sync> --threads 8 --history 64 --cache-mb 64
```

Each run writes one JSON file with every measurement in this report. `Comparison report` turns a set of them into Markdown tables. The tables, figures and quoted numbers in this report were generated from the same files by `benchmarks/report/gen.py`, without manual transcription. The dataset statistics in table 2 are printed by `io.hstore.bench.DatasetStats`, run with the same classpath. The raw files are in `benchmarks/report/results`, and the report is rebuilt with:

```
cd benchmarks/report
python3 gen.py results gen
tectonic report.tex
```

D HStore settings

Setting	Meaning	Value
`page_size`	tree node page size	16 KiB
segment size	pages per segment file	16,384 (256 MiB)
`node_cache_mb`	decoded node cache	64 MiB (set)
`write_buffer_mb`	cap on unwritten tree nodes	64 MiB
`checkpoint_wal_mb`	log or page bytes that trigger a checkpoint	256 MiB
`compaction_live_ratio`	compact segments below this live fraction	0.5
`history_limit`	generations kept for time travel	64
`wal_mode`	log page references, not page images	references
maintenance interval	background checkpoint and compaction pass	500 ms